大模型推理优化讲义

一、推理阶段的两类瓶颈

大模型的推理分为预填充阶段和解码阶段。预填充阶段一次性处理整个输入序列，计算密集，GPU 利用率高；解码阶段逐 token 生成，每生成一个 token 都要读取全部模型权重和缓存，属于访存密集。长上下文场景下，解码阶段的显存占用和访存开销会成为主要瓶颈。

二、KV cache 的显存开销

自回归生成时，每一层都要缓存历史 token 的键和值，避免重复计算。KV cache 的显存占用可以用公式估算：2 × 层数 × 注意力头数 × 头维度 × 序列长度 × 批大小 × 数据精度字节数。以 7B 模型、32 层、32 个头、头维度 128、FP16 为例，单个 token 的 KV cache 约为 0.5MB，三十二 K 上下文、批大小 8 时仅缓存就超过 128GB。因此长上下文推理必须对 KV cache 做压缩或分页管理。

三、FlashAttention

标准注意力需要显式构造 N×N 的注意力矩阵，显存占用随序列长度平方增长。FlashAttention 把注意力计算切分成适合 SRAM 的小块，在块内完成 softmax 的在线归一化，全程不把完整注意力矩阵写入显存。这样显存占用从 $O(N^2)$ 降到 O(N)，同时因为减少了显存读写，实际速度也更快。FlashAttention 不改变计算结果，属于精确优化，不是近似方法。

四、KV cache 压缩策略

第一类是窗口淘汰：只保留最近 W 个 token 的缓存，配合注意力汇点保留开头少量 token，避免分布偏移。第二类是量化：把 FP16 的缓存压到 INT8 或 INT4，显存直接减半或更多，但需要处理离群值。第三类是低秩分解：把键值投影到低维空间。第四类是动态稀疏：根据注意力分数保留重要的 token。分页注意力把缓存切成固定大小的页，按需分配，解决变长序列导致的内存碎片问题。

这些策略的共同代价是精度损失或额外计算。选择时要在显存、延迟和效果之间做权衡，并通过评测集验证掉点是否可接受。

五、吞吐与延迟的权衡

提高吞吐的常用手段是增大批大小和连续批处理，把不同请求的 token 拼在同一批里计算。但批越大，单次解码的延迟越高，KV cache 占用也越大。在线服务通常用 P99 延迟作为约束，再在约束内尽量提高吞吐。量化、蒸馏和投机解码则从模型侧降低单 token 成本。

理解这些取舍，比记住某个框架的参数更重要。